

How It Works

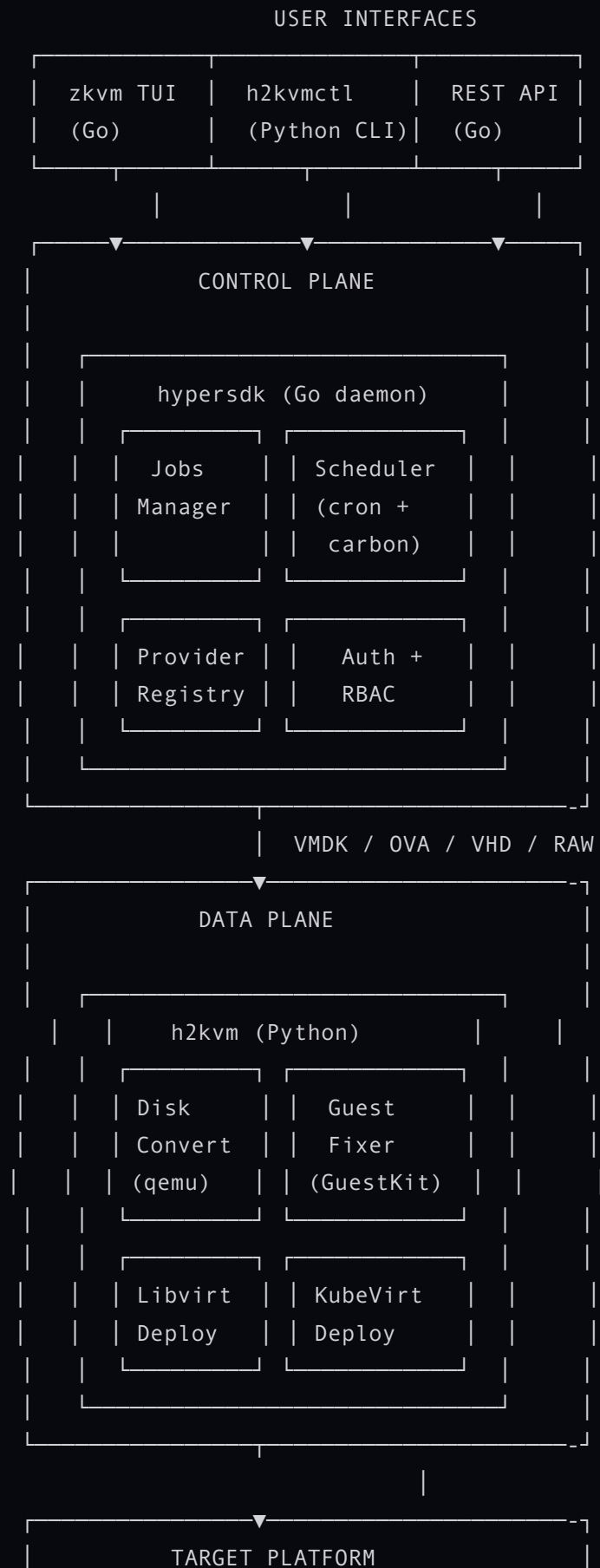
hypersdk + h2kvm Technical Deep Dive

A two-layer platform: Go daemon for multi-cloud export + Python engine for disk conversion and guest repair. This document shows exactly how every component works together.

For architects, engineers, and technical evaluators

System Overview

Two independently deployable layers that work together seamlessly.



| KVM/libvirt | KubeVirt/Kubernetes |

Go

hypersdk — Control Plane

Multi-cloud VM discovery and export. REST API, job scheduling, RBAC, cost estimation, carbon-aware scheduling. Hexagonal architecture with 13+ port interfaces.

Python

h2kvm — Data Plane

Disk conversion, offline guest OS repair, libvirt XML generation, KubeVirt deployment. 1,273 tests. GuestKit engine for filesystem manipulation.



hypersdk Architecture

Hexagonal architecture — domain, ports, and adapters.

Domain Layer (internal/domain/) — 70+ Types

File	Key Types	Purpose
job.go	Job, JobStatus, JobProgress, Session	Export job lifecycle
schedule.go	ScheduledJob, RetryPolicy, TimeWindow	Cron + time-window scheduling
provider.go	ProviderType, VMInfo, ExportOptions	Cloud provider abstraction
cost.go	CostEstimate, PricingData	Migration cost calculation
carbon.go	CarbonIntensity, CarbonForecast	Green IT / ESG scheduling
rbac.go	Role, Permission, Action, Resource	Access control
audit.go	AuditEvent, AuditQueryFilter	Compliance audit trail
secrets.go	Secret, SecretType	Credential management
webhook.go	Webhook, WebhookPayload	Event notifications

Ports Layer (internal/ports/) — 13+ Interfaces

```
type ComputeProvider interface {
    ListVMs(ctx context.Context, filter VMFilter) ([]VMInfo, error)
    GetVM(ctx context.Context, id string) (*VMInfo, error)
}

type ExportProvider interface {
    ExportVM(ctx context.Context, vmID string, opts ExportOptions) (*ExportResult, error)
    GetCapabilities() []ExportCapability
}

type JobManager interface {
    CreateJob(def JobDefinition) (*Job, error)
    GetJob(id string) (*Job, error)
    ListJobs(filter JobFilter) ([]*Job, error)
    CancelJob(id string) error
}

type CarbonScheduler interface {
    FindOptimalWindow(job *Job, forecast []CarbonForecast) (*TimeWindow, error)
}
```

Cloud Providers

Each provider implements ComputeProvider + ExportProvider interfaces.

Provider	Discovery	Export Method	Auth
VMware vSphere	govc / pyvmomi / SOAP API	OVF export, OVA, HTTPS / folder, CBT delta	SSO, user/pass
AWS EC2	EC2 DescribeInstances	EBS snapshot → S3 → download, AMI export	IAM, access keys
Microsoft Azure	Compute client	Managed disk snapshot → blob → download	Service principal, MSI
Google Cloud	Compute API	Disk snapshot → GCS → download	Service account JSON
Hyper-V	WinRM / PowerShell	VHD/VHDX direct copy, checkpoint export	WinRM, SSH
OCI	Core client	Boot volume → object storage → download	API key, config file
OpenStack	Nova list-servers	Glance image download, Cinder snapshot	Keystone v3
Alibaba Cloud	ECS DescribeInstances	Disk snapshot → OSS → download	AccessKey ID/Secret
Proxmox VE	REST API /nodes/qemu	API-based disk download, qcow2/raw	Token, PAM
KubeVirt	kubectl get vm	PVC/DataVolume export, CDI clone	kubeconfig

8 Binary Components

hyperexport

Standalone interactive VM export. Select VMs, choose format, export with progress. Works without daemon.

hypervisord

Background daemon. REST API on port 9292. Job queue, scheduling, webhooks. Runs as systemd service.

hyperctl

CLI migration commander. Create jobs, manage schedules, query status. Like kubectl for migrations.

hyperconvert

Format conversion tool. VMDK/VHD/RAW/ qcow2 with progress. Standalone or daemon-integrated.

hypersdk-agent

gRPC agent for distributed export. Deploy near source infrastructure. Streams data to control plane.

hypersdk-control

Control plane for multi-agent deployments. Manages agent fleet, routes jobs, aggregates results.

hypersdk-dashboard

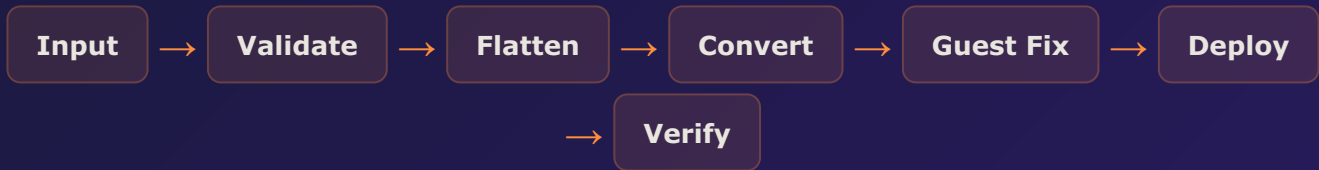
Web UI for monitoring. Job status, metrics, cost reports. Built-in Prometheus + Grafana integration.

hypersdk-operator

Kubernetes operator. CRDs for migration jobs. GitOps-friendly. Helm chart included.

Python h2kvm Pipeline

What happens to a VM disk from raw export to running on KVM.



Python Package Structure

```
h2kvm/
├── orchestration/          # Pipeline orchestrator – chains all stages
│   ├── orchestrator.py    # Main pipeline: validate → convert → fix → deploy
│   ├── disk_processor.py  # Per-disk processing with retry
│   └── vsphere_exporter.py # vSphere export integration
├── converters/            # Format conversion (qemu-img wrapper)
├── fixers/                # Offline guest OS repair
│   ├── offline_vm/        # Linux fixes: fstab, initramfs, grub, selinux
│   └── windows/           # Windows: VirtIO injection, registry, BCD
├── providers/
│   ├── vmware/            # vSphere client, govc wrapper, NFC lease
│   └── azure/             # Azure disk export
├── libvirt/               # Domain XML generation, virsh integration
├── pipeline/              # KubeVirt deployment, PVC upload
├── cli/                   # Argument parser, formatters, progress bars
├── config/                # YAML config loading, validation
├── quality/               # VMDK inspector, pre-flight checks, testing
├── ai/                    # Migration intelligence (risk analysis)
└── core/                  # Logging, exceptions, structured tracing
```

GuestKit: Guest Fix Engine

The critical differentiator — why VMs boot on first try.



fstab Stabilization

Scans /etc/fstab inside the disk image. Converts device-name entries (/dev/sda1) to UUID-based entries. Handles LVM, XFS UUID regeneration, BTRFS subvolumes. Three modes: stabilize-all, bypath-only, noop.



initramfs Rebuild

Chroots into guest OS. Injects virtio_blk, virtio_net, virtio_scsi kernel modules. Runs dracut/mkinitramfs to rebuild. Supports RHEL, Ubuntu, Debian, SUSE, Photon OS.



GRUB Repair

Updates root= kernel parameter to match new device path. Regenerates grub.cfg. Handles both BIOS and UEFI boot. Fixes broken grubenv symlinks.



SELinux Relabel

Creates /.autorelabel trigger file. On first boot, SELinux relabels all files with correct contexts. Prevents boot failures from label mismatches.



Windows VirtIO

Mounts virtio-win.iso. Injects viostor (storage), NetKVM (network), Balloon (memory) drivers. Updates Windows registry to load VirtIO at boot. Multi-stage: bootstrap → drivers → cleanup.



Cleanup

Removes VMware Tools (open-vm-tools). Removes Hyper-V integration services. Clears stale network configs (VMXNET3 MAC bindings). Reduces attack surface.

How GuestKit Works (Under the Hood)

```
# 1. Connect to disk via qemu-nbd (no libguestfs needed)
sudo qemu-nbd --connect=/dev/nbd0 disk.qcow2
```

```
# 2. Detect partitions and mount
partprobe /dev/nbd0
mount /dev/nbd0p2 /mnt/guest      # root partition
mount /dev/nbd0p1 /mnt/guest/boot # boot partition
```

```
# 3. Chroot and fix
```

```
chroot /mnt/guest /bin/bash
# Fix fstab (UUID stabilization)
# Rebuild initramfs (dracut --force --add virtio)
# Update GRUB (grub2-mkconfig)
# Create /.autorelabel

# 4. Unmount and disconnect
umount -R /mnt/guest
qemu-nbd --disconnect /dev/nbd0
```

zkvm Interface

Go + Bubble Tea terminal UI for guided migration.

Configuration

▼ Step 1: Source source

/data/vm/server.vmdk

Detected: VMDK (VMware)

▼ Step 2: Output

./out

◀ [qcow2] ▶

▼ Step 4: Deploy to Libvirt

● ON

Status

Status

● READY – Press Ctrl+R to run

Execution Plan

✓ Input: VMDK (VMware)

→ Convert: qcow2

→ Define: libvirt XML

→ Test: boot verify

Deploy Targets

[✓] Libvirt XML (1)

[✓] virsh define (2)

[✓] Boot test (3)

[] Kubernetes (4)

Shortcuts

1-4 Toggle deploy

Ctrl+R Run migration

Tab Switch tabs

1 Migration | 2 vSphere | 3 Logs | 4 Libvirt VMs | 5 Kubernetes

5 Tabs

Migration (guided form), vSphere (VM discovery + batch select), Logs (scrollable), Libvirt VMs (manage), Kubernetes (KubeVirt VMs)

File Browser

Built-in fzf-style fuzzy file picker. Filters by extension. Navigate dirs, search instantly. Opens on Enter for path fields.

vSphere Discovery

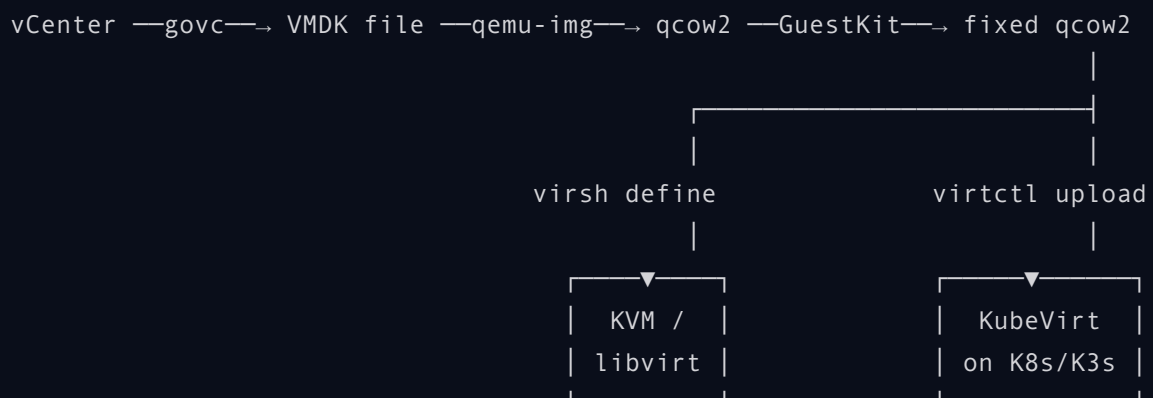
Auto-populate from GOVC_* env vars. Batch discover VMs with CPU/RAM/power. Select multiple, queue migrate with Ctrl+R.

End-to-End Flow

What happens when you migrate a VM from VMware vSphere to KubeVirt.

Step	Component	What Happens	Time
1. Discover	<code>Go</code> hypersdk / govc	Connect to vCenter via SOAP API. List VMs with <code>govc ls + vm.info -json</code> . Return CPU, RAM, disk size, power state.	5-15s
2. Export	<code>Go</code> hypersdk	<code>govc export.ovf</code> downloads VMDK via <code>HttpNfcLease</code> . Fallback: OVA, then <code>HTTPS /folder</code> . Progress streamed via <code>\r</code> lines.	5-60 min
3. Validate	<code>Py</code> quality/	Parse VMDK descriptor. Check extent sizes. Detect boot mode (BIOS/UEFI) via <code>virt-inspector</code> . Risk analysis (adapter type, geometry).	2-5s
4. Flatten	<code>Py</code> converters/	<code>qemu-img convert</code> merges snapshot chain into single self-contained <code>qcow2</code> . Progress reported at 1% intervals.	1-10 min
5. Guest Fix	<code>Py</code> fixers/	Mount via <code>qemu-nbd</code> . Fix <code>fstab</code> (UUID). Rebuild <code>initramfs</code> (<code>virtio</code>). Fix GRUB. Remove VMware tools. SELinux relabel trigger.	30-120s
6. Deploy	<code>Py</code> libvirt/ + pipeline/	Generate libvirt domain XML (machine=q35, UEFI if detected). <code>virsh define</code> . OR: create KubeVirt <code>VirtualMachine</code> + PVC, upload via <code>virtctl</code> .	5-30s
7. Verify	<code>Py</code> quality/	Start VM in KVM. Wait for boot. Check no kernel panic. Verify network. Report pass/fail.	30-60s

Data Flow



Deployment Options

Multiple ways to run the platform.



CLI (h2kvmctl)

Single command or YAML config. Best for scripts, CI/CD, cron jobs. No daemon needed.

```
sudo h2kvmctl --config migration.yaml
```



TUI (zkvm)

Interactive terminal UI. Guided steps, file browser, vSphere discovery. Best for operators.

```
./zkvm/zkvm
```



K8s Operator

CRDs for migration jobs. Helm chart. GitOps-friendly. Best for K8s-native teams.

```
helm install hypersdk ./helm/hypersdk
```



Container

OCI image for Podman/Docker. Mount disks as volumes. Best for CI/CD pipelines.

```
podman run -v ./:/data hypersdk convert
```

hypersdk + h2kvm | Proprietary (Zyvor AI Labs) Open Source | github.com/zyvorai/h2kvm

API Examples

How to integrate programmatically.

YAML Config (h2kvmctl)

```
# Full vSphere → KubeVirt migration
cmd: vsphere
vcenter: vcenter.corp.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
dc_name: DC1
vs_action: export_vm
vs_vm: production-web-01

output_dir: ./out
out_format: qcow2
flatten: true
regen_initramfs: true
fstab_mode: stabilize-all
emit_domain_xml: true

deploy_k8s: true
k8s_vm_name: web-01
k8s_namespace: production
k8s_pvc_size: 20Gi
k8s_memory: 4Gi
k8s_cpu: 4
```

REST API (hypervisord)

```
# Create export job
curl -X POST http://localhost:9292/api/v1/jobs \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "provider": "vsphere",
    "vm_id": "vm-123",
    "export_format": "qcow2",
    "output_dir": "/exports",
    "schedule": "0 2 * * SAT"
  }'

# Check job status
curl http://localhost:9292/api/v1/jobs/job-abc-123

# List VMs from provider
```

```
curl http://localhost:9292/api/v1/providers/vsphere/vms
```

Kubernetes CRD

```
apiVersion: hypersdk.io/v1alpha1
kind: HyperConversion
metadata:
  name: migrate-web-01
spec:
  source:
    provider: vsphere
    vmName: production-web-01
  target:
    type: kubevirt
    namespace: production
    pvcSize: 20Gi
  fixes:
    fstabMode: stabilize-all
    regenInitramfs: true
```